就诊管理模块讲解文档

一、数据库与实体类设计

1.1 核心实体表设计

1.1.1 就诊记录实体表(medical_record)

```sql
CREATE TABLE `medical_record` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT '记录ID',
  `record_no` varchar(20) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NOT
NULL COMMENT '记录编号',
  `patient_id` bigint NOT NULL COMMENT '患者ID',
  `doctor_id` bigint NOT NULL COMMENT '医生ID',
  `appointment_id` bigint NULL DEFAULT NULL COMMENT '预约ID',
  `diagnosis` text CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NULL COMMENT
'诊断结果',
  `treatment` text CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NULL COMMENT
'治疗方案',
  `record_date` date NOT NULL COMMENT '就诊日期',
  `notes` varchar(500) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NULL
DEFAULT NULL COMMENT '医生备注',
  `follow_up` date NULL DEFAULT NULL COMMENT '随访日期',
  `create_time` datetime NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  `update_time` datetime NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP COMMENT '更新时间',
  PRIMARY KEY (`id`) USING BTREE,
  INDEX `idx_patient_id`(`patient_id` ASC) USING BTREE,
  INDEX `idx_doctor_id`(`doctor_id` ASC) USING BTREE,
  INDEX `idx_appointment_id`(`appointment_id` ASC) USING BTREE
) ENGINE = InnoDB AUTO_INCREMENT = 1 CHARACTER SET = utf8mb4 COLLATE =
utf8mb4_0900_ai_ci COMMENT = '就诊记录表' ROW_FORMAT = Dynamic;
```

MedicalRecord实体类:

```java
@Data
@TableName("medical_record")
@Schema(description = "就诊记录实体类")
public class MedicalRecord {
    @TableId(type = IdType.AUTO)
    @Schema(description = "记录ID")
    private Long id;

    @Schema(description = "记录编号")
    private String recordNo;

    @Schema(description = "患者ID")
    private Long patientId;

    @Schema(description = "医生ID")
    private Long doctorId;

    @Schema(description = "预约ID")
```

半夜撕代码版权所有

```java
    private Long appointmentId;

    @Schema(description = "诊断结果")
    private String diagnosis;

    @Schema(description = "治疗方案")
    private String treatment;

    @Schema(description = "就诊日期")
    private LocalDate recordDate;

    @Schema(description = "医生备注")
    private String notes;

    @Schema(description = "随访日期")
    private LocalDate followUp;

    @Schema(description = "创建时间")
    private LocalDateTime createTime;

    @Schema(description = "更新时间")
    private LocalDateTime updateTime;

    @TableField(exist = false)
    @Schema(description = "患者信息")
    private Patient patient;

    @TableField(exist = false)
    @Schema(description = "医生信息")
    private Doctor doctor;

    @TableField(exist = false)
    @Schema(description = "预约信息")
    private Appointment appointment;
}
```

1.1.2 处方实体表(prescription)

```mysql
CREATE TABLE `prescription` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT '处方ID',
  `prescription_no` varchar(20) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci
NOT NULL COMMENT '处方编号',
  `patient_id` bigint NOT NULL COMMENT '患者ID',
  `doctor_id` bigint NOT NULL COMMENT '医生ID',
  `record_id` bigint NOT NULL COMMENT '就诊记录ID',
  `prescription_date` date NOT NULL COMMENT '处方日期',
  `diagnosis` varchar(500) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NULL
DEFAULT NULL COMMENT '诊断',
  `notes` varchar(500) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NULL
DEFAULT NULL COMMENT '备注',
  `status` tinyint NULL DEFAULT 0 COMMENT '状态(0:未取药,1:已取药)',
  `create_time` datetime NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  `update_time` datetime NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP COMMENT '更新时间',
  PRIMARY KEY (`id`) USING BTREE,
```

半夜撕代码版权所有

```
  INDEX `idx_patient_id`(`patient_id` ASC) USING BTREE,
  INDEX `idx_doctor_id`(`doctor_id` ASC) USING BTREE,
  INDEX `idx_record_id`(`record_id` ASC) USING BTREE
) ENGINE = InnoDB AUTO_INCREMENT = 1 CHARACTER SET = utf8mb4 COLLATE =
utf8mb4_0900_ai_ci COMMENT = '处方表' ROW_FORMAT = Dynamic;
```

Prescription实体类:

```
@Data
@TableName("prescription")
@Schema(description = "处方实体类")
public class Prescription {
    @TableId(type = IdType.AUTO)
    @Schema(description = "处方ID")
    private Long id;

    @Schema(description = "处方编号")
    private String prescriptionNo;

    @Schema(description = "患者ID")
    private Long patientId;

    @Schema(description = "医生ID")
    private Long doctorId;

    @Schema(description = "就诊记录ID")
    private Long recordId;

    @Schema(description = "处方日期")
    private LocalDate prescriptionDate;

    @Schema(description = "诊断")
    private String diagnosis;

    @Schema(description = "备注")
    private String notes;

    @Schema(description = "状态(0:未取药,1:已取药)")
    private Integer status;

    @Schema(description = "创建时间")
    private LocalDateTime createTime;

    @Schema(description = "更新时间")
    private LocalDateTime updateTime;

    @TableField(exist = false)
    @Schema(description = "患者信息")
    private Patient patient;

    @TableField(exist = false)
    @Schema(description = "医生信息")
    private Doctor doctor;

    @TableField(exist = false)
```

半夜撕代码版权所有

```
    @Schema(description = "就诊记录信息")
    private MedicalRecord medicalRecord;

    @TableField(exist = false)
    @Schema(description = "处方明细列表")
    private List<PrescriptionDetail> details;
}
```

1.2 核心属性说明

1.2.1 就诊记录实体核心属性

- **id**: 就诊记录的唯一标识符，自增长
- **recordNo**: 就诊记录编号，系统自动生成，格式为"MR" + 年月日 + 6位随机数
- **patientId**: 关联患者ID，表示该就诊记录属于哪个患者
- **doctorId**: 关联医生ID，表示该就诊记录由哪个医生创建
- **appointmentId**: 关联预约ID，可为空，表示该就诊记录是否由预约转化而来
- **diagnosis**: 诊断结果，文本格式
- **treatment**: 治疗方案，文本格式
- **recordDate**: 就诊日期，日期格式
- **notes**: 医生备注，文本格式
- **followUp**: 随访日期，可为空，日期格式
- **patient**: 非数据库字段，关联患者信息
- **doctor**: 非数据库字段，关联医生信息
- **appointment**: 非数据库字段，关联预约信息

1.2.2 处方实体核心属性

- **id**: 处方的唯一标识符，自增长
- **prescriptionNo**: 处方编号，系统自动生成
- **patientId**: 关联患者ID，表示该处方属于哪个患者
- **doctorId**: 关联医生ID，表示该处方由哪个医生开具
- **recordId**: 关联就诊记录ID，表示该处方属于哪次就诊
- **prescriptionDate**: 处方日期，日期格式
- **diagnosis**: 诊断，文本格式
- **notes**: 备注，文本格式
- **status**: 状态，0表示未取药，1表示已取药
- **details**: 非数据库字段，处方明细列表，包含药品信息

半夜撕代码版权所有

二、各功能详细讲解

2.1 就诊记录管理功能

2.1.1 功能概述

就诊记录管理功能是医院信息系统的核心功能之一，它允许医生记录患者的就诊情况，包括诊断结果、治疗方案、随访安排等信息。系统支持创建、查询、编辑和删除就诊记录，并提供权限控制，确保医生只能操作自己的就诊记录。同时，系统还支持将预约转化为就诊记录，提高工作效率。

2.1.2 详细讲解功能实现流程

1. **创建就诊记录流程**：
   医生在前端vue3/src/views/backend/medical-record/index.vue页面点击"新增就诊记录"按钮，弹出对话框，填写就诊信息后，点击提交按钮，调用submitForm方法，该方法通过request.post向/medical-record接口发送请求。后端MedicalRecordController的createMedicalRecord方法接收请求，首先检查医生权限，然后调用medicalRecordService.createMedicalRecord方法，该方法检查患者和医生是否存在，生成就诊记录编号，设置创建时间和更新时间，最后将就诊记录保存到数据库。如果有关联预约，还会更新预约状态为已就诊。
2. **查询就诊记录流程**：
   前端通过onMounted钩子调用fetchMedicalRecords方法，该方法使用request.get向/medical-record/page接口发送请求，传递查询参数。后端MedicalRecordController的getMedicalRecordsByPage方法接收请求，检查当前用户是否是医生，如果是医生，则只能查看自己的就诊记录，然后调用medicalRecordService.getMedicalRecordsByPage方法进行查询，该方法使用MyBatis-Plus的LambdaQueryWrapper构建动态查询条件，最后将查询结果返回给前端。
3. **编辑就诊记录流程**：
   医生在就诊记录列表中点击"编辑"按钮，弹出对话框，修改就诊信息后，点击提交按钮，调用submitForm方法，该方法通过request.put向/medical-record/{id}接口发送请求。后端MedicalRecordController的updateMedicalRecord方法接收请求，首先检查医生权限，然后调用medicalRecordService.updateMedicalRecord方法，该方法检查就诊记录是否存在，设置更新时间，最后更新就诊记录。
4. **删除就诊记录流程**：
   医生在就诊记录列表中点击"删除"按钮，弹出确认对话框，点击确认后，调用handleDelete方法，该方法通过request.delete向/medical-record/{id}接口发送请求。后端MedicalRecordController的deleteMedicalRecord方法接收请求，首先检查医生权限，然后调用medicalRecordService.deleteMedicalRecord方法，该方法检查就诊记录是否存在，检查是否有关联处方，如果没有则删除就诊记录。
5. **患者查看就诊记录流程**：
   患者在前端vue3/src/views/frontend/medical-record/index.vue页面，通过onMounted钩子调用fetchMyMedicalRecords方法，该方法使用request.get向/medical-record/page接口发送请求，传递患者ID参数。后端返回患者的就诊记录列表，前端还会调用loadPrescriptions方法，获取每个就诊记录关联的处方信息，最终在页面上展示完整的就诊记录和处方信息。

2.1.3 关键代码讲解

1. **创建就诊记录代码**
   【springboot/src/main/java/org/example/springboot/service/MedicalRecordService.java：createMedicalRecord方法】

```
@Transactional
```

半夜撕代码版权所有

```java
public MedicalRecord createMedicalRecord(MedicalRecord medicalRecord) {
    // 检查患者是否存在
    Patient patient = patientMapper.selectById(medicalRecord.getPatientId());
    if (patient == null) {
        throw new ServiceException("患者不存在");
    }

    // 检查医生是否存在
    Doctor doctor = doctorMapper.selectById(medicalRecord.getDoctorId());
    if (doctor == null) {
        throw new ServiceException("医生不存在");
    }

    // 检查权限，如果是医生，只能创建自己的就诊记录
    checkDoctorPermission(medicalRecord);

    // 如果有关联预约，检查预约是否存在
    if (medicalRecord.getAppointmentId() != null) {
        Appointment appointment =
appointmentMapper.selectById(medicalRecord.getAppointmentId());
        if (appointment == null) {
            throw new ServiceException("预约不存在");
        }

        // 更新预约状态为已就诊
        if (appointment.getStatus() == 1) {
            appointmentService.completeAppointment(appointment.getId());
        }
    }

    // 生成就诊记录编号
    medicalRecord.setRecordNo(generateRecordNo());

    // 如果没有指定就诊日期，使用当前日期
    if (medicalRecord.getRecordDate() == null) {
        medicalRecord.setRecordDate(LocalDate.now());
    }

    // 设置创建时间和更新时间
    LocalDateTime now = LocalDateTime.now();
    medicalRecord.setCreateTime(now);
    medicalRecord.setUpdateTime(now);

    if (medicalRecordMapper.insert(medicalRecord) <= 0) {
        throw new ServiceException("就诊记录创建失败");
    }

    return medicalRecord;
}
```

这段代码实现了创建就诊记录的业务逻辑。首先检查患者和医生是否存在，然后检查医生权限，确保只能创建自己的就诊记录。如果有关联预约，还会检查预约是否存在，并更新预约状态为已就诊。接着生成就诊记录编号，设置默认就诊日期、创建时间和更新时间，最后将就诊记录保存到数据库。

半夜撕代码版权所有

2. **医生权限检查代码**
【springboot/src/main/java/org/example/springboot/service/MedicalRecordService.java：checkDoctorPermission方法】

```
public void checkDoctorPermission(MedicalRecord medicalRecord) {
    Long currentUserId = JwtTokenUtils.getCurrentUserId();
    if (currentUserId == null) {
        return; // 未登录，由其他权限控制处理
    }

    try {
        Doctor currentDoctor =
doctorService.getDoctorByUserId(currentUserId);
        if (currentDoctor != null &&
!currentDoctor.getId().equals(medicalRecord.getDoctorId())) {
            throw new ServiceException("您只能操作自己的就诊记录");
        }
    } catch (ServiceException e) {
        throw e;
    } catch (Exception e) {
        // 不是医生，忽略异常
    }
}
```

这段代码实现了医生权限检查的逻辑。它首先获取当前登录用户的ID，然后尝试获取该用户对应的医生信息。如果当前用户是医生，则检查就诊记录的doctorId是否与当前医生ID相同，如果不同则抛出异常，表示医生只能操作自己的就诊记录。如果当前用户不是医生（例如是管理员），则忽略异常，允许操作。

3. **分页查询就诊记录代码**
【springboot/src/main/java/org/example/springboot/service/MedicalRecordService.java：getMedicalRecordsByPage方法】

```
public Page<MedicalRecord> getMedicalRecordsByPage(Long patientId, Long
doctorId, String patientName, String doctorName,
                                                   LocalDate startDate, LocalDate
endDate, Integer currentPage, Integer size) {
    Page<MedicalRecord> page = new Page<>(currentPage, size);

    // 如果有患者姓名或医生姓名查询条件，需要进行关联查询
    if (StringUtils.isNotBlank(patientName) ||
StringUtils.isNotBlank(doctorName)) {
        // 使用自定义SQL查询
        Page<MedicalRecord> resultPage =
medicalRecordMapper.selectMedicalRecordsByNamePage(
            page, patientId, doctorId, patientName, doctorName, startDate,
endDate);

        // 填充关联信息
        fillMedicalRecordsInfo(resultPage.getRecords());

        return resultPage;
    }
```

半夜撕代码版权所有

```
    LambdaQueryWrapper<MedicalRecord> queryWrapper = new LambdaQueryWrapper<>
();

    // 添加查询条件
    if (patientId != null) {
        queryWrapper.eq(MedicalRecord::getPatientId, patientId);
    }
    if (doctorId != null) {
        queryWrapper.eq(MedicalRecord::getDoctorId, doctorId);
    }
    if (startDate != null) {
        queryWrapper.ge(MedicalRecord::getRecordDate, startDate);
    }
    if (endDate != null) {
        queryWrapper.le(MedicalRecord::getRecordDate, endDate);
    }

    // 按就诊日期降序排序
    queryWrapper.orderByDesc(MedicalRecord::getRecordDate);

    Page<MedicalRecord> resultPage = medicalRecordMapper.selectPage(page,
queryWrapper);

    // 填充关联信息
    fillMedicalRecordsInfo(resultPage.getRecords());

    return resultPage;
}
```

这段代码实现了分页查询就诊记录的业务逻辑。它支持两种查询方式：1) 如果有患者姓名或医生姓名查询条件，则使用自定义SQL进行关联查询；2) 否则使用MyBatis-Plus的LambdaQueryWrapper构建动态查询条件。查询条件包括患者ID、医生ID、开始日期、结束日期等。查询结果按就诊日期降序排序，并填充关联的患者和医生信息。

4. **删除就诊记录前检查代码**
【springboot/src/main/java/org/example/springboot/service/MedicalRecordService.java：deleteMedicalRecord方法】

```
@Transactional
public void deleteMedicalRecord(Long id) {
    // 检查就诊记录是否存在
    MedicalRecord medicalRecord = medicalRecordMapper.selectById(id);
    if (medicalRecord == null) {
        throw new ServiceException("就诊记录不存在");
    }

    // 检查权限，如果是医生，只能删除自己的就诊记录
    checkDoctorPermission(medicalRecord);

    // 检查就诊记录是否有关联处方
    LambdaQueryWrapper<Prescription> queryWrapper = new LambdaQueryWrapper<>
();
    queryWrapper.eq(Prescription::getRecordId, id);
    long count = prescriptionMapper.selectCount(queryWrapper);
    if (count > 0) {
```

半夜撕代码版权所有

```
        throw new ServiceException("该就诊记录已关联处方，不能删除");
    }

    if (medicalRecordMapper.deleteById(id) <= 0) {
        throw new ServiceException("就诊记录删除失败");
    }
}
```

这段代码实现了删除就诊记录前的安全检查逻辑。它首先检查就诊记录是否存在，然后检查医生权限，确保只能删除自己的就诊记录。接着查询处方表，检查就诊记录是否有关联处方，如果有则不允许删除，避免数据不一致。最后，如果通过所有检查，则删除就诊记录。

2.1.4 功能实现细节

1. **就诊记录编号生成**：系统自动生成就诊记录编号，格式为"MR" + 年月日 + 6位随机数，确保唯一性。
2. **医生权限控制**：系统实现了严格的权限控制，确保医生只能创建、查看、编辑和删除自己的就诊记录，避免越权操作。
3. **关联预约处理**：创建就诊记录时，如果关联了预约，系统会自动更新预约状态为已就诊，实现业务流程的闭环。
4. **关联处方保护**：删除就诊记录前，系统会检查是否有关联处方，如果有则不允许删除，保护数据完整性。
5. **关联信息填充**：查询就诊记录时，系统会自动填充关联的患者和医生信息，方便前端展示完整的就诊记录数据。

2.2 患者就诊记录查询功能

2.2.1 功能概述

患者就诊记录查询功能允许患者查看自己的历史就诊记录，包括诊断结果、治疗方案、随访安排以及关联的处方信息。系统提供了友好的用户界面，支持按医生姓名和就诊日期范围搜索，并以时间轴形式展示就诊记录，方便患者了解自己的健康状况变化。

2.2.2 详细讲解功能实现流程

1. **患者查看就诊记录流程**：
   患者登录系统后，在前端vue3/src/views/frontend/medical-record/index.vue页面，通过onMounted钩子调用fetchMyMedicalRecords方法，该方法使用request.get向/medical-record/page接口发送请求，传递患者ID参数。后端MedicalRecordController的getMedicalRecordsByPage方法接收请求，调用medicalRecordService.getMedicalRecordsByPage方法进行查询，返回患者的就诊记录列表。前端收到响应后，还会调用loadPrescriptions方法，向/prescription/record/{recordId}接口发送请求，获取每个就诊记录关联的处方信息，最终在页面上以时间轴形式展示完整的就诊记录和处方信息。
2. **就诊记录详情查看流程**：
   患者在就诊记录列表中点击"查看详情"按钮，调用handleViewRecord方法，该方法将当前就诊记录赋值给currentRecord变量，并显示就诊记录详情对话框。详情对话框展示就诊记录的完整信息，包括基本信息、诊断结果、治疗方案、随访日期以及关联的处方信息。

半夜撕代码版权所有

3. **处方详情查看流程**：
   患者在就诊记录详情中点击"查看处方详情"按钮，调用handleViewPrescription方法，该方法将当前处方赋值给currentPrescription变量，并显示处方详情对话框。处方详情对话框展示处方的完整信息，包括基本信息、药品明细等。

2.2.3 关键代码讲解

1. **获取患者就诊记录代码**
   【vue3/src/views/frontend/medical-record/index.vue：fetchMyMedicalRecords方法】

```
const fetchMyMedicalRecords = async () => {
  loading.value = true
  try {
    await request.get('/medical-record/page', {
      patientId: patientId.value,
      doctorName: searchForm.doctorName,
      startDate: searchForm.startDate,
      endDate: searchForm.endDate,
      currentPage: currentPage.value,
      size: pageSize.value
    }, {
      onSuccess: async (res) => {
        // 获取就诊记录
        medicalRecords.value = res.records || []
        total.value = res.total || 0

        // 为每个就诊记录加载处方信息
        for (const record of medicalRecords.value) {
          await loadPrescriptions(record)
        }
      }
    })
  } catch (error) {
    console.error('获取就诊记录失败:', error)
    ElMessage.error('获取就诊记录失败')
  } finally {
    loading.value = false
  }
}
```

   这段代码实现了获取患者就诊记录的前端逻辑。它首先设置loading状态，然后向后端接口发送GET请求，传递患者ID、医生姓名、日期范围等参数。请求成功后，将返回的就诊记录列表赋值给medicalRecords变量，并调用loadPrescriptions方法加载每个就诊记录关联的处方信息。最后，无论请求成功还是失败，都会重置loading状态。

2. **加载处方信息代码**
   【vue3/src/views/frontend/medical-record/index.vue：loadPrescriptions方法】

半夜撕代码版权所有

```
const loadPrescriptions = async (record) => {
  try {
    await request.get(`/prescription/record/${record.id}`, {}, {
      onSuccess: (res) => {
        record.prescriptions = res || []
      }
    })
  } catch (error) {
    console.error('获取处方信息失败:', error)
  }
}
```

这段代码实现了加载就诊记录关联的处方信息的逻辑。它向后端接口发送GET请求，传递就诊记录ID，请求成功后，将返回的处方列表赋值给record.prescriptions变量，这样就诊记录对象就包含了关联的处方信息，方便前端展示。

3. **时间轴展示就诊记录代码**
【vue3/src/views/frontend/medical-record/index.vue：时间轴组件】

```
<el-timeline>
  <el-timeline-item
    v-for="record in medicalRecords"
    :key="record.id"
    :timestamp="record.recordDate"
    placement="top"
    :type="getTimelineItemType(record)"
  >
    <el-card class="record-card">
      <template #header>
        <div class="record-header">
          <div class="record-title">
            <span class="record-no">{{ record.recordNo }}</span>
            <el-tag size="small" effect="plain">{{
record.doctor?.department?.deptName || '未知科室' }}</el-tag>
          </div>
          <div class="record-actions">
            <el-button type="primary" size="small"
@click="handleViewRecord(record)">查看详情</el-button>
          </div>
        </div>
      </template>

      <el-descriptions :column="2" border size="small">
        <el-descriptions-item label="就诊医生">
          {{ record.doctor?.name || '未知' }}
          <el-tag size="small">{{ record.doctor?.title || '' }}</el-tag>
        </el-descriptions-item>
        <el-descriptions-item label="就诊日期">{{ record.recordDate }}</el-
descriptions-item>
        <el-descriptions-item label="诊断结果" :span="2">{{ record.diagnosis
|| '暂无诊断' }}</el-descriptions-item>
        <el-descriptions-item label="治疗方案" :span="2">{{ record.treatment
|| '暂无治疗方案' }}</el-descriptions-item>
```

```
        <el-descriptions-item label="随访日期" v-if="record.followUp">{{
record.followUp }}</el-descriptions-item>
      </el-descriptions>
    </el-card>
  </el-timeline-item>
</el-timeline>
```

这段代码实现了使用时间轴组件展示就诊记录的前端界面。它遍历medicalRecords数组，为每个就诊记录创建一个时间轴项，显示就诊日期、就诊医生、诊断结果、治疗方案等信息。时间轴项的类型由getTimelineItemType方法决定，根据随访日期设置不同的颜色，提高用户体验。每个就诊记录卡片还包含一个"查看详情"按钮，点击后可以查看更详细的就诊信息。

4. **时间轴项类型计算代码**
【vue3/src/views/frontend/medical-record/index.vue：getTimelineItemType方法】

```
const getTimelineItemType = (record) => {
  if (record.followUp) {
    const today = new Date()
    const followUpDate = new Date(record.followUp)

    if (followUpDate > today) {
      return 'warning' // 即将随访
    } else if (followUpDate.toDateString() === today.toDateString()) {
      return 'success' // 今天随访
    }
  }

  return 'primary' // 默认类型
}
```

这段代码实现了计算时间轴项类型的逻辑。它根据就诊记录的随访日期，决定时间轴项的显示类型：如果随访日期在今天之后，则显示为"warning"类型，表示即将随访；如果随访日期是今天，则显示为"success"类型，表示今天随访；其他情况显示为"primary"类型。这种设计可以直观地提示患者关注即将到来的随访安排。

2.2.4 功能实现细节

1. **就诊记录与处方关联**：系统在查询就诊记录后，会自动加载每个就诊记录关联的处方信息，提供完整的就诊信息。
2. **时间轴可视化**：前端使用时间轴组件，按时间顺序展示就诊记录，并根据随访日期设置不同的颜色，提高用户体验。
3. **详情对话框**：系统提供就诊记录详情和处方详情对话框，展示更详细的信息，方便患者了解自己的健康状况。
4. **搜索功能**：系统支持按医生姓名和就诊日期范围搜索，方便患者快速找到特定的就诊记录。

2.3 处方管理功能

2.3.1 功能概述

处方管理功能是就诊管理模块的重要组成部分，它允许医生为患者开具处方，记录药品用法用量，并支持处方状态管理。系统支持创建、查询、编辑和删除处方，并提供权限控制，确保医生只能操作自己的处方。患者可以查看自己的处方信息，了解药品用法用量，提高用药安全性。

半夜撕代码版权所有

2.3.2 详细讲解功能实现流程

1. **创建处方流程**：
   医生在就诊记录详情页面点击"新增处方"按钮，弹出处方编辑对话框，填写处方信息后，点击提交按钮，调用submitPrescriptionForm方法，该方法通过request.post向/prescription接口发送请求。后端PrescriptionController的createPrescription方法接收请求，首先检查医生权限，然后调用prescriptionService.createPrescription方法，该方法检查患者、医生和就诊记录是否存在，生成处方编号，设置创建时间和更新时间，最后将处方保存到数据库。
2. **添加处方明细流程**：
   创建处方后，医生可以添加处方明细，包括药品、用量、频次、用药天数等信息。前端通过request.post向/prescription/detail接口发送请求。后端PrescriptionController的addPrescriptionDetail方法接收请求，首先检查医生权限，然后调用prescriptionService.addPrescriptionDetail方法，该方法检查处方和药品是否存在，设置创建时间和更新时间，最后将处方明细保存到数据库。
3. **更新处方状态流程**：
   药房人员在处方列表中点击"已取药"按钮，调用handleUpdateStatus方法，该方法通过request.put向/prescription/status/{id}接口发送请求。后端PrescriptionController的updatePrescriptionStatus方法接收请求，首先检查医生权限，然后调用prescriptionService.updatePrescriptionStatus方法，该方法检查处方是否存在，设置更新时间，最后更新处方状态。
4. **患者查看处方流程**：
   患者在就诊记录详情页面可以查看关联的处方信息，点击"查看处方详情"按钮，弹出处方详情对话框，展示处方的基本信息和药品明细。前端通过request.get向/prescription/{id}接口发送请求，获取处方详情。

2.3.3 关键代码讲解

1. **创建处方代码**
   【springboot/src/main/java/org/example/springboot/controller/PrescriptionController.java：createPrescription方法】

```
@Operation(summary = "新增处方")
@PostMapping
public Result<?> createPrescription(@RequestBody Prescription prescription) {
    LOGGER.info("新增处方: patientId={}, doctorId={}, recordId={}",
            prescription.getPatientId(), prescription.getDoctorId(),
prescription.getRecordId());

    // 检查权限，如果是医生，只能创建自己的处方
    checkDoctorPermission(prescription);

    Prescription newPrescription =
prescriptionService.createPrescription(prescription);
    return Result.success(newPrescription);
}
```

   这段代码实现了创建处方的控制器方法。它首先记录日志，然后调用checkDoctorPermission方法检查医生权限，确保只能创建自己的处方。接着调用prescriptionService.createPrescription方法创建处方，最后返回成功结果和新创建的处方对象。

半夜撕代码版权所有

2. **医生权限检查代码**
【springboot/src/main/java/org/example/springboot/controller/PrescriptionController.java：checkDoctorPermission方法】

```
private void checkDoctorPermission(Prescription prescription) {
    Long currentUserId = JwtTokenUtils.getCurrentUserId();
    if (currentUserId == null) {
        return; // 未登录，由其他权限控制处理
    }

    try {
        Doctor currentDoctor =
doctorService.getDoctorByUserId(currentUserId);
        if (currentDoctor != null &&
!currentDoctor.getId().equals(prescription.getDoctorId())) {
            throw new ServiceException("您只能操作自己的处方");
        }
    } catch (ServiceException e) {
        throw e;
    } catch (Exception e) {
        // 不是医生，忽略异常
    }
}
```

这段代码实现了医生权限检查的逻辑。它首先获取当前登录用户的ID，然后尝试获取该用户对应的医生信息。如果当前用户是医生，则检查处方的doctorId是否与当前医生ID相同，如果不同则抛出异常，表示医生只能操作自己的处方。如果当前用户不是医生（例如是管理员），则忽略异常，允许操作。

3. **获取就诊记录关联的处方列表代码**
【springboot/src/main/java/org/example/springboot/controller/PrescriptionController.java：getPrescriptionsByRecord方法】

```
@Operation(summary = "获取就诊记录关联的处方列表")
@GetMapping("/record/{recordId}")
public Result<?> getPrescriptionsByRecord(@PathVariable Long recordId) {
    return
Result.success(prescriptionService.getPrescriptionsByRecord(recordId));
}
```

这段代码实现了获取就诊记录关联的处方列表的控制器方法。它接收就诊记录ID作为参数，调用prescriptionService.getPrescriptionsByRecord方法获取处方列表，最后返回成功结果和处方列表。这个方法通常用于在就诊记录详情页面展示关联的处方信息。

2.3.4 功能实现细节

1. **处方与就诊记录关联**：系统通过recordId字段将处方与就诊记录关联起来，方便查询和管理。
2. **处方状态管理**：系统支持处方状态管理，包括未取药和已取药两种状态，方便跟踪处方执行情况。
3. **处方明细管理**：系统支持添加、编辑和删除处方明细，记录药品用法用量，提高用药安全性。
4. **医生权限控制**：系统实现了严格的权限控制，确保医生只能创建、查看、编辑和删除自己的处方，避免越权操作。

半夜撕代码版权所有

2.4 就诊记录与预约的关联功能

2.4.1 功能概述

就诊记录与预约的关联功能实现了预约到就诊的业务流程闭环。当患者完成预约并到医院就诊时，医生可以通过选择关联的预约创建就诊记录，系统会自动更新预约状态为已就诊，避免重复操作，提高工作效率。

2.4.2 详细讲解功能实现流程

1. **预约关联就诊记录流程**：
   医生创建就诊记录时，可以在下拉框中选择患者的预约，系统会自动填充患者信息。提交就诊记录后，系统会自动更新预约状态为已就诊，完成预约到就诊的流程闭环。
2. **预约状态更新流程**：
   当创建就诊记录并关联预约时，系统会调用appointmentService.completeAppointment方法，将预约状态更新为已就诊（status=2），表示该预约已完成就诊流程。

2.4.3 关键代码讲解

1. **预约关联就诊记录代码**
   【springboot/src/main/java/org/example/springboot/service/MedicalRecordService.java：createMedicalRecord方法】

   ```
   // 如果有关联预约，检查预约是否存在
   if (medicalRecord.getAppointmentId() != null) {
       Appointment appointment =
   appointmentMapper.selectById(medicalRecord.getAppointmentId());
       if (appointment == null) {
           throw new ServiceException("预约不存在");
       }

       // 更新预约状态为已就诊
       if (appointment.getStatus() == 1) {
           appointmentService.completeAppointment(appointment.getId());
       }
   }
   ```

   这段代码实现了预约关联就诊记录的业务逻辑。当创建就诊记录时，如果指定了appointmentId，系统会检查该预约是否存在。如果预约存在且状态为待就诊（status=1），则调用appointmentService.completeAppointment方法，将预约状态更新为已就诊，完成预约到就诊的流程闭环。
2. **完成预约代码**
   【springboot/src/main/java/org/example/springboot/service/AppointmentService.java：completeAppointment方法】

   ```
   @Transactional
   public void completeAppointment(Long id) {
       // 检查预约是否存在
       Appointment appointment = getAppointmentById(id);

       // 更新预约状态为已就诊
       Appointment updateAppointment = new Appointment();
       updateAppointment.setId(id);
   ```

半夜撕代码版权所有

```
    updateAppointment.setStatus(2); // 已就诊
    updateAppointment.setUpdateTime(LocalDateTime.now());

    if (appointmentMapper.updateById(updateAppointment) <= 0) {
        throw new ServiceException("预约状态更新失败");
    }

    // 如果有关联排班，更新排班的当前预约人数
    if (appointment.getScheduleId() != null) {
        scheduleService.updateSchedulePatients(appointment.getScheduleId(),
-1);
    }
}
```

这段代码实现了完成预约的业务逻辑。它首先检查预约是否存在，然后更新预约状态为已就诊（status=2），并设置更新时间。如果预约关联了排班，还会调用scheduleService.updateSchedulePatients方法，减少排班的当前预约人数，释放排班名额。

2.4.4 功能实现细节

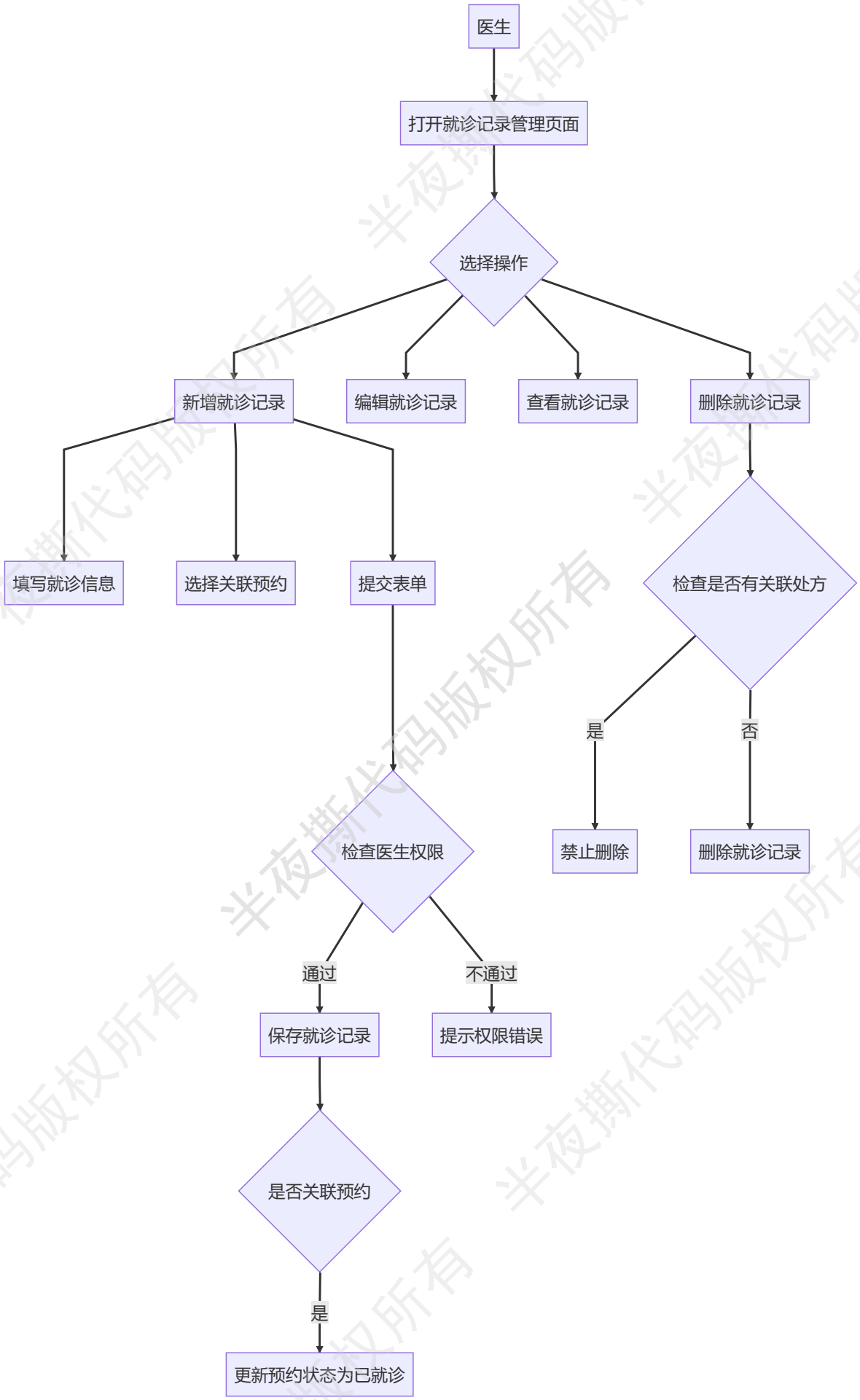
1. **预约选择界面**：创建就诊记录时，系统提供预约选择下拉框，显示患者的待就诊预约，方便医生关联预约。
2. **预约状态管理**：系统支持预约状态管理，包括取消、待就诊和已就诊三种状态，完整跟踪预约生命周期。
3. **预约到就诊的流程闭环**：通过关联预约创建就诊记录，并自动更新预约状态，实现预约到就诊的业务流程闭环。

三、总结

3.1 就诊记录管理功能实现

就诊记录管理功能通过MedicalRecordController和MedicalRecordService实现。医生可以在后台管理页面创建、查询、编辑和删除就诊记录，并记录诊断结果、治疗方案、随访安排等信息。系统实现了严格的权限控制，确保医生只能操作自己的就诊记录。创建就诊记录时，可以关联预约，系统会自动更新预约状态为已就诊，实现预约到就诊的业务流程闭环。删除就诊记录前，系统会检查是否有关联处方，如果有则不允许删除，保护数据完整性。

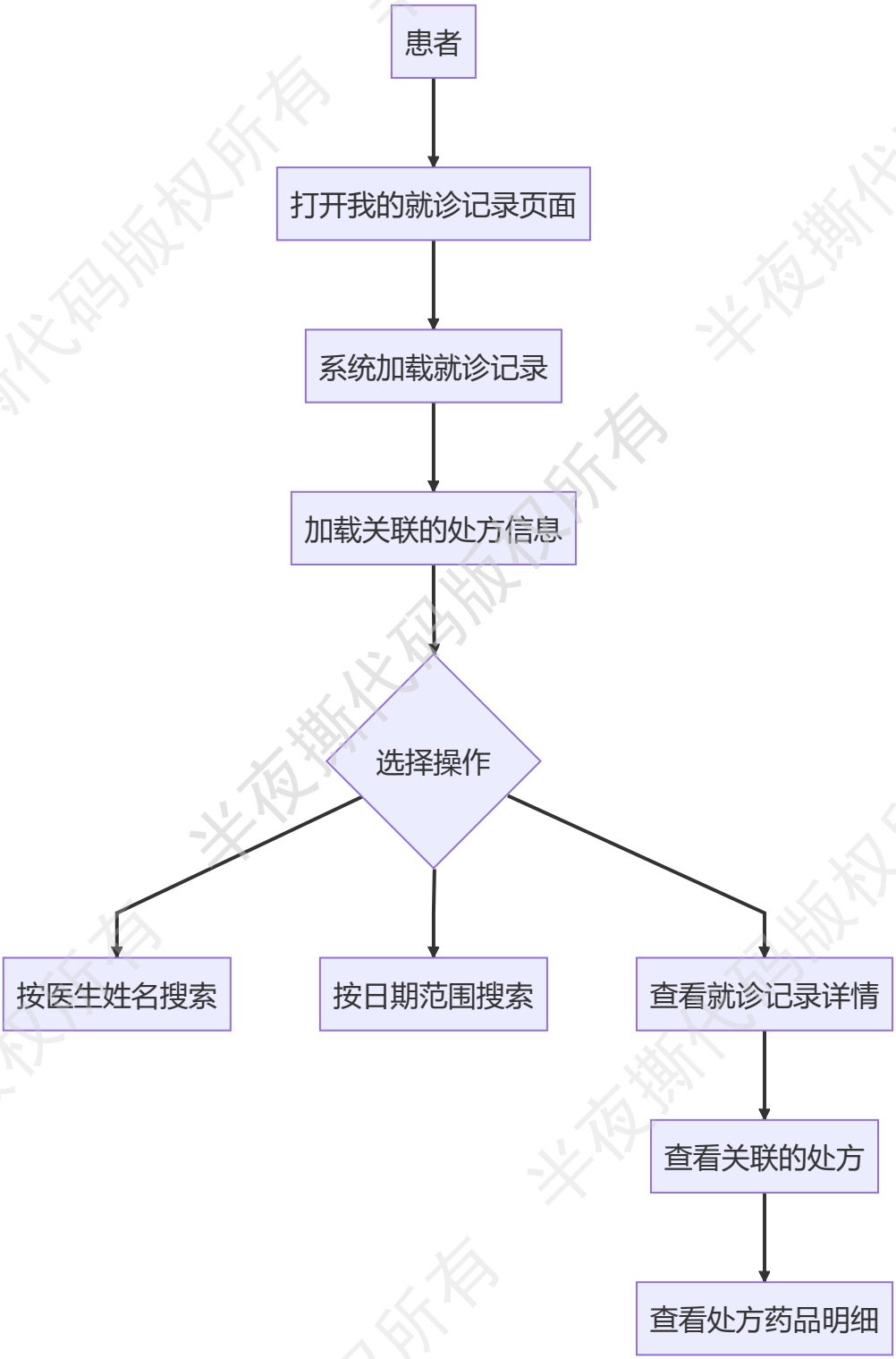
半夜撕代码版权所有


医生
打开就诊记录管理页面
选择操作
新增就诊记录
编辑就诊记录
查看就诊记录
删除就诊记录
填写就诊信息
选择关联预约
提交表单
检查是否有关联处方
是
否
禁止删除
删除就诊记录
检查医生权限
通过
不通过
保存就诊记录
提示权限错误
是否关联预约
是
更新预约状态为已就诊

半夜撕代码版权所有

3.2 患者就诊记录查询功能实现

患者就诊记录查询功能通过前端vue3/src/views/frontend/medical-record/index.vue页面和后端MedicalRecordController实现。患者可以在前端页面查看自己的历史就诊记录，包括诊断结果、治疗方案、随访安排以及关联的处方信息。系统提供了友好的用户界面，支持按医生姓名和就诊日期范围搜索，并以时间轴形式展示就诊记录，方便患者了解自己的健康状况变化。点击就诊记录可以查看详细信息，包括关联的处方和药品明细，提高用药安全性。

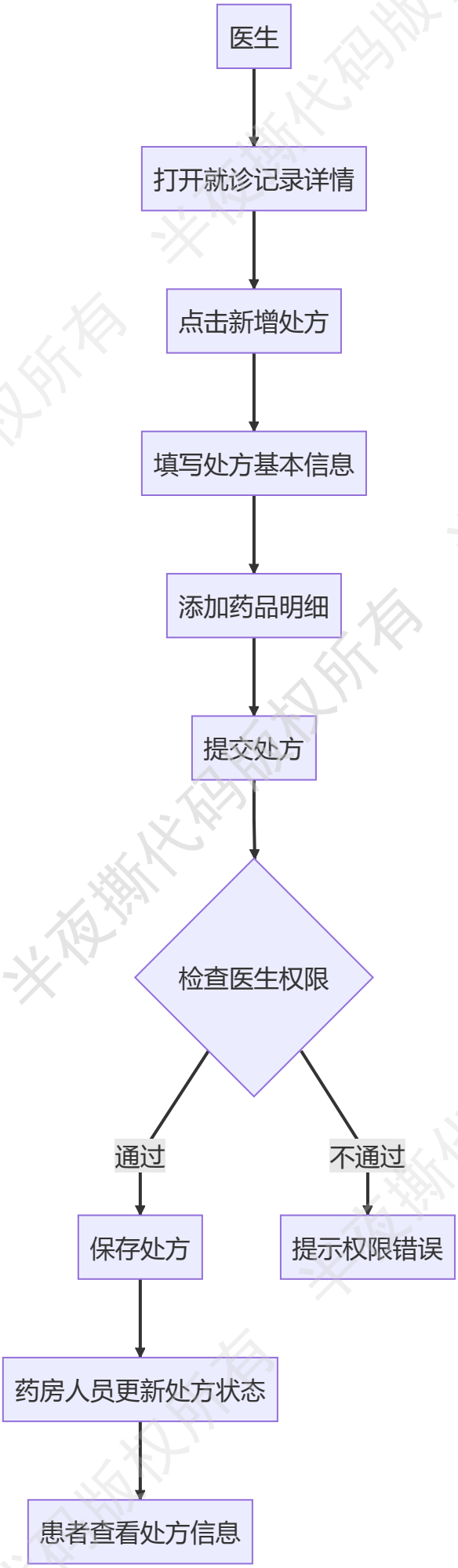


半夜撕代码版权所有

3.3 处方管理功能实现

处方管理功能通过PrescriptionController和PrescriptionService实现。医生可以在就诊记录详情页面为患者开具处方，添加药品明细，记录用法用量。系统实现了严格的权限控制，确保医生只能操作自己的处方。药房人员可以更新处方状态为已取药，跟踪处方执行情况。患者可以在就诊记录详情页面查看自己的处方信息，了解药品用法用量，提高用药安全性。

半夜撕代码版权所有


医生
打开就诊记录详情
点击新增处方
填写处方基本信息
添加药品明细
提交处方
检查医生权限
通过
不通过
保存处方
提示权限错误
药房人员更新处方状态
患者查看处方信息

半夜撕代码版权所有

3.4 就诊记录与预约的关联功能实现

就诊记录与预约的关联功能实现了预约到就诊的业务流程闭环。当患者完成预约并到医院就诊时，医生可以通过选择关联的预约创建就诊记录，系统会自动更新预约状态为已就诊，避免重复操作，提高工作效率。同时，系统还会更新排班的当前预约人数，释放排班名额，方便其他患者预约。

就诊管理模块是医院信息系统的核心功能之一，它实现了患者从预约到就诊、开具处方的完整业务流程，提供了全面的就诊信息管理功能，方便医生记录患者的就诊情况，也方便患者了解自己的健康状况。通过严格的权限控制和数据完整性检查，确保系统数据的安全性和一致性。

半夜撕代码版权所有